中国科学院大学
University of Chinese Academy of Sciences

# Design and Development of a 3D Printed Self-Solving Rubik's Cube Robot

**Intelligence Software Engineer Course Project**


**Team Members:**

Mohamed Ishag Hassan

Jerry Wang

Atinkut Molla Mekonnen

**Supervisor:** Professor Luo Tiejian

University of Chinese Academy of Sciences


Date: 17/12/2025

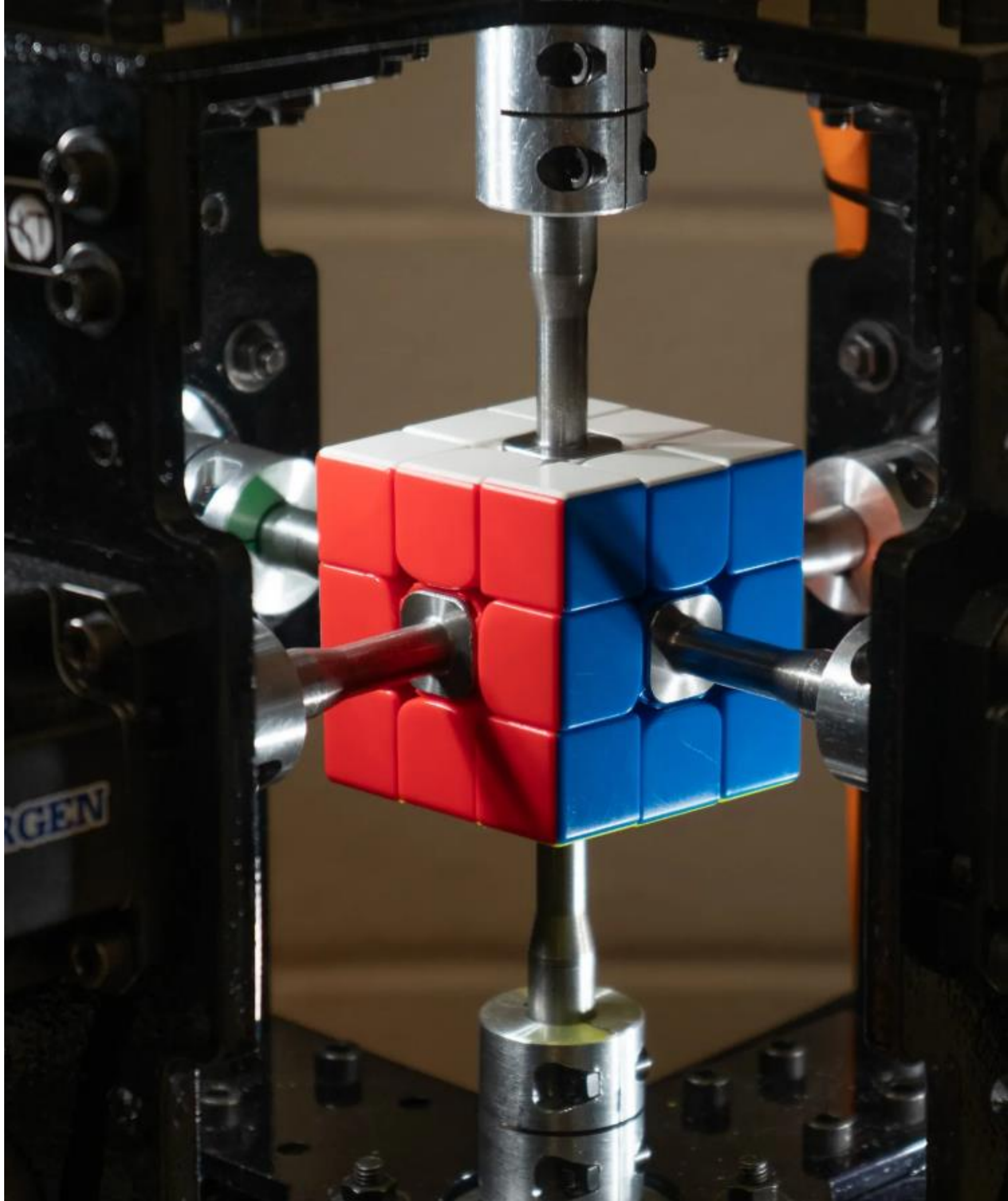
Table of Contents

# 1. Introduction

1. This project transcends the traditional task of puzzle-solving by framing the Rubik's Cube as a complex mechanical environment for Autonomous Sensorimotor Discovery. Rather than a mere toy, the cube is defined as a high-dimensional state-space system where a robotic agent must learn to navigate through physical interactions.

2. Our framework utilizes a custom-engineered 3D-printed apparatus equipped with four stepper motors, serving as the Experimental Apparatus. Controlled by an Arduino Mega 2560, the system facilitates a self-supervised learning loop where the robot executes low-level action primitives to observe and model state transitions.

3. This approach demonstrates the integration of affordable additive manufacturing with advanced robotics, shifting the focus from "hard-coded solving" to "grounded physical modeling." The ultimate goal is to validate a low-cost prototype capable of mapping unknown transition functions in a physical environment.



## 2. Formal Problem Definition

- The Rubik's Cube is not just a puzzle, but a physical system with formal mathematical structure:

**The Rubik’s Cube as a Formal State-Space System**

- Mathematical Framework: State Space: The set of all possible physical configurations of the cube (approx. 4.3 x 10^19 states).

| Symbol Meaning In Our System | Symbol Meaning In Our System | Symbol Meaning In Our System |
|---|---|---|
| S | State Space All possible color configurations ($\approx 4.3\times10^{19}$ states) | State Space All possible color configurations ($\approx 4.3\times10^{19}$ states) |
| A | Action Space | $\{U, U', U^2, R, R', R^2, F, F', F^2, L, L', L^2, D, D', D^2, B, B', B^2\}$ |
| T | Transition Function Physical result of applying action $a$ in state (s) | Transition Function Physical result of applying action $a$ in state $s$ |
| s-0 | Initial State | Scrambled cube configuration |
| s-g | Goal State | Solved cube (all faces uniform) |

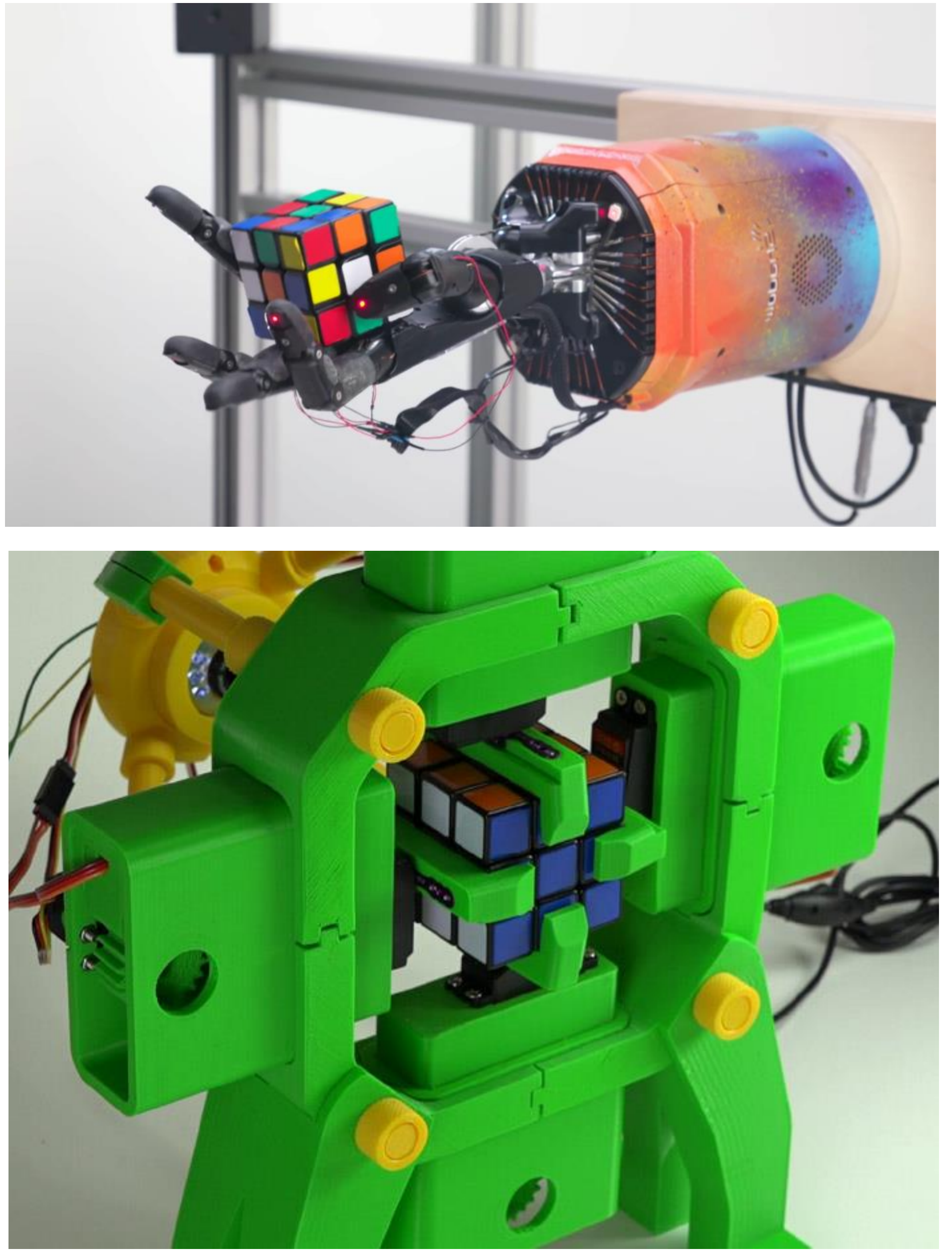


Fig.1. Different Self solving robot

Equation 1: System Dynamics $s_{t+1} = T(s_t, a_t)$

Where (T) is the physical transition function learned through experimentation.

$$\hat{T}_\theta(s, a) \approx T(s, a)$$

Equation 2: Learning Objective

Find parameters (theta) that approximate the true physics through robotic interaction

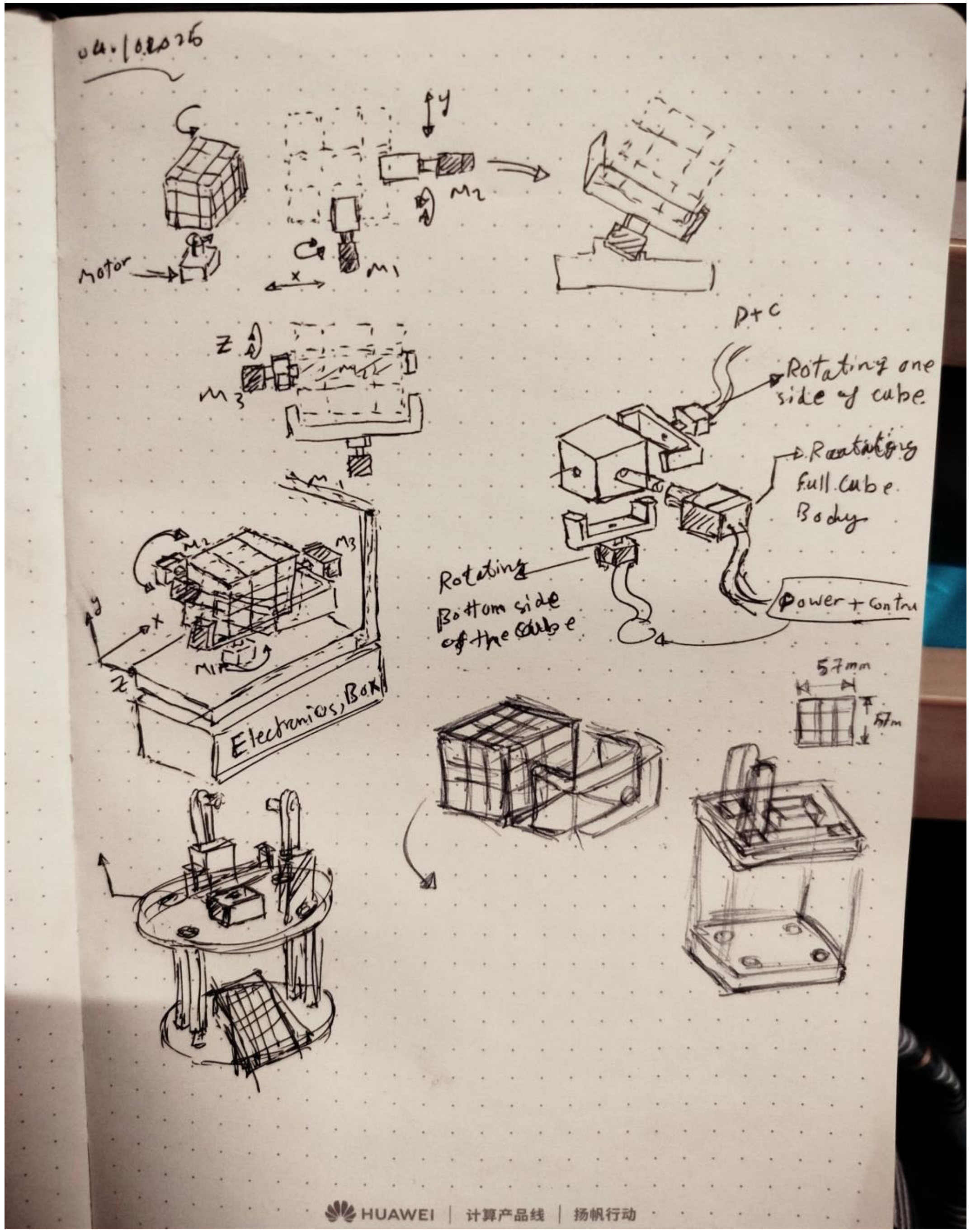
Transition Function ($T$): The unknown physical mapping where T: $S \times A \to S.$

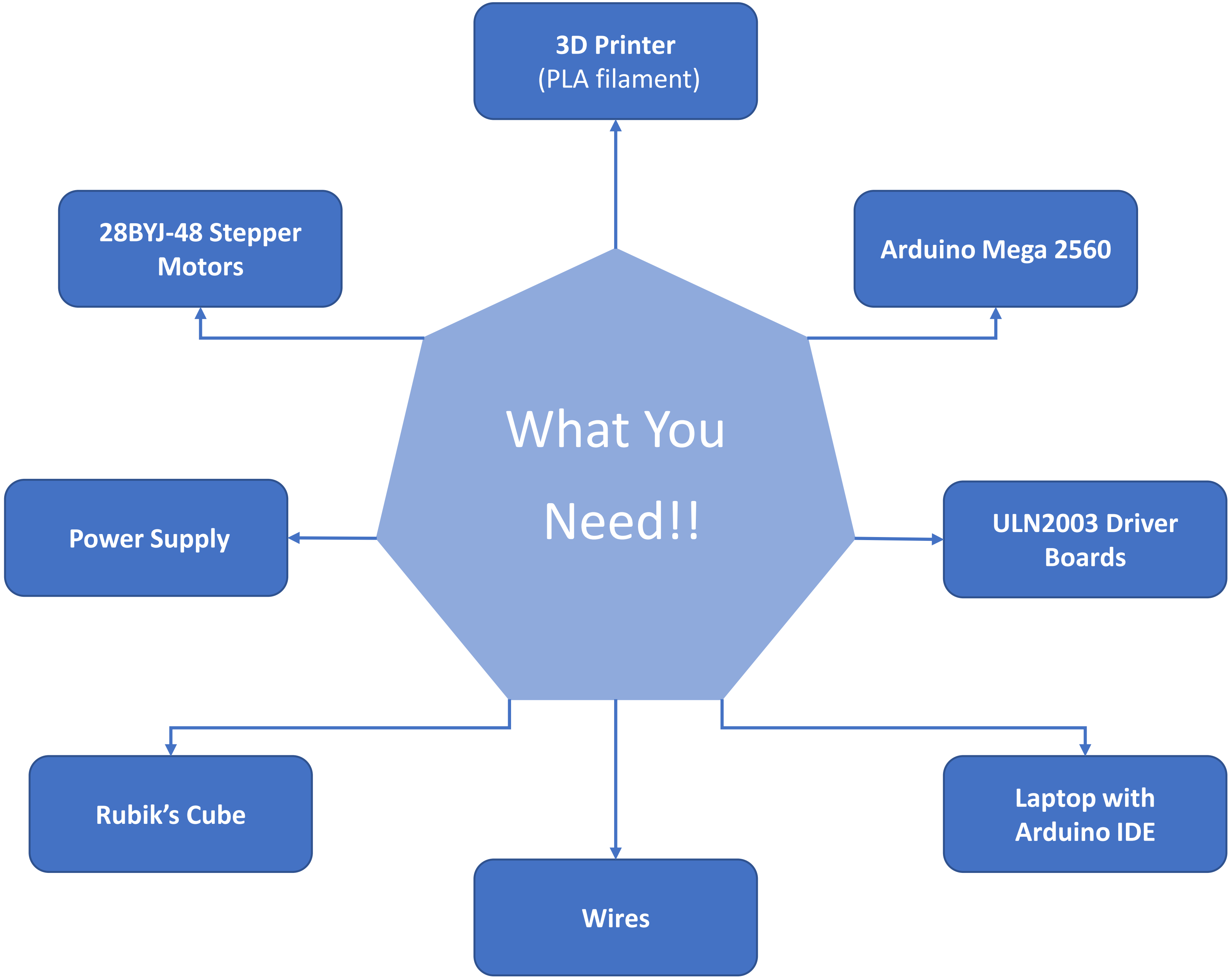
The Learning Objective: To train a neural network model M that approximates the transition function T:

$$M(s_t, a_t) \approx s_{t+1}.$$

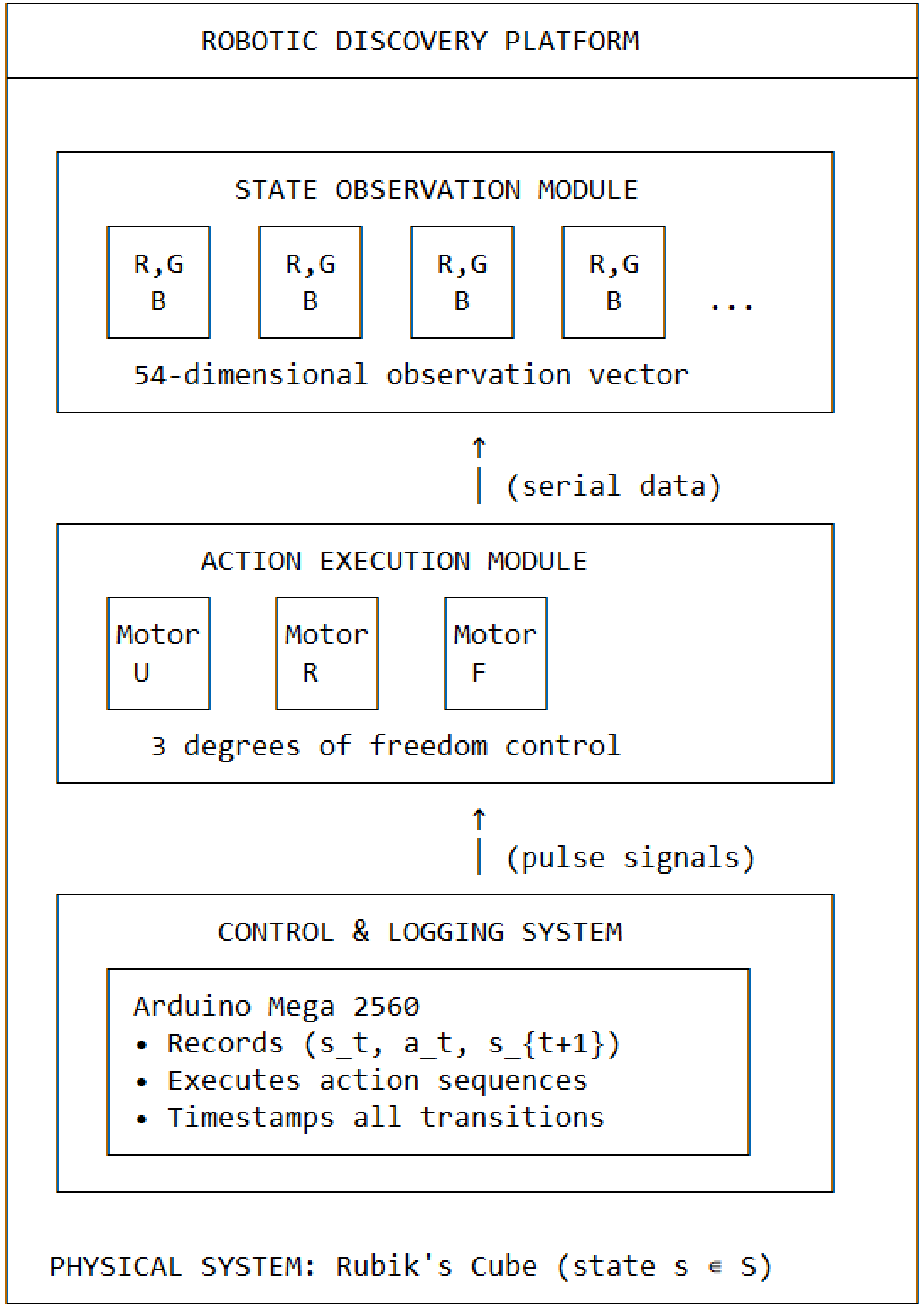

Fig.2. Mathematics Rubik’s Cube

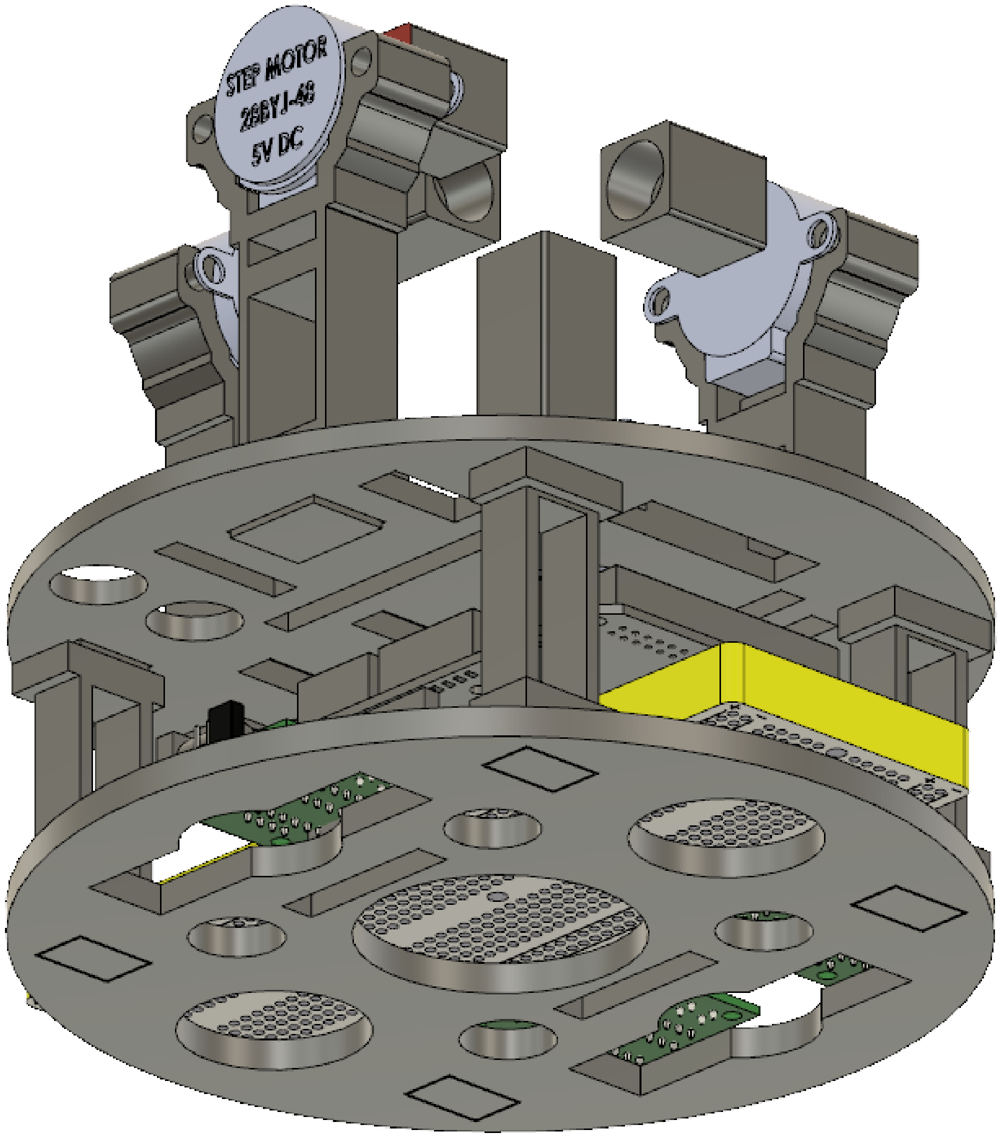
3. Solution


Motor
M1
M2
M3
P+C
Rotating one side of cube
Rotating full Cube Body
Rotating Bottom side of the cube
Power + Contro
Electronics Box
57mm
HUAWEI | 计算产品线 | 扬帆行动



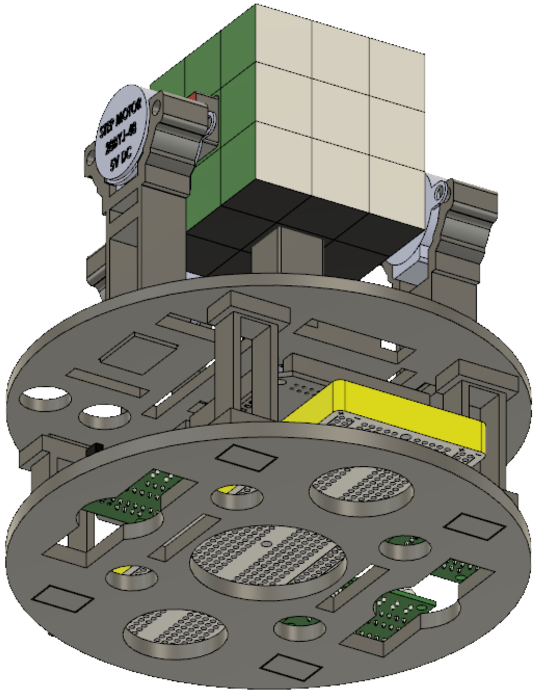
What You Need!!
3D Printer (PLA filament)
Arduino Mega 2560
ULN2003 Driver Boards
Laptop with Arduino IDE
Wires
Rubik's Cube
Power Supply
28BYJ-48 Stepper Motors

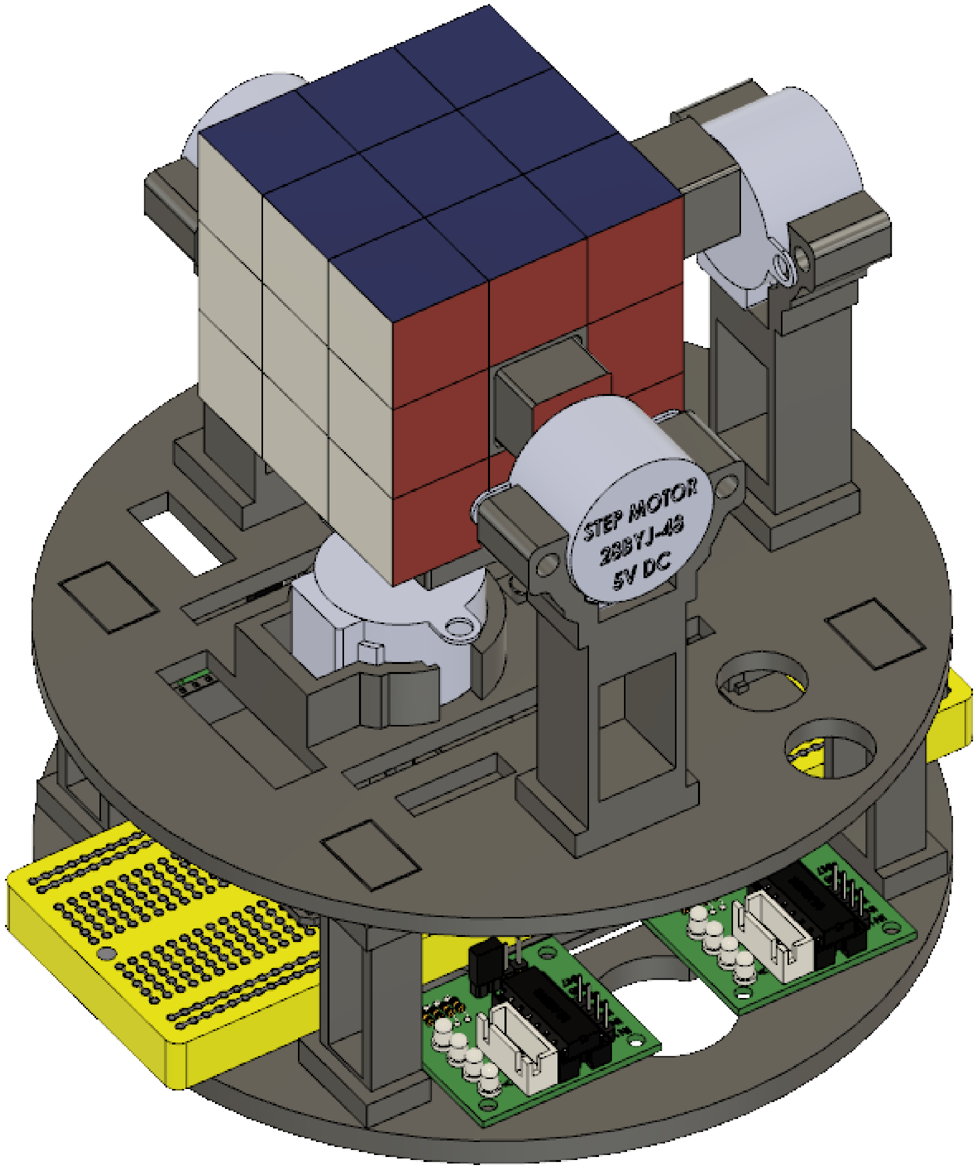

ROBOTIC DISCOVERY PLATFORM
STATE OBSERVATION MODULE
R,G B
R,G B
R,G B
R,G B
...
54-dimensional observation vector
(serial data)
ACTION EXECUTION MODULE
Motor U
Motor R
Motor F
3 degrees of freedom control
(pulse signals)
CONTROL & LOGGING SYSTEM
Arduino Mega 2560
• Records (s_t, a_t, s_{t+1})
• Executes action sequences
• Timestamps all transitions
PHYSICAL SYSTEM: Rubik's Cube (state s ∊ S)


STEP MOTOR
5V DC

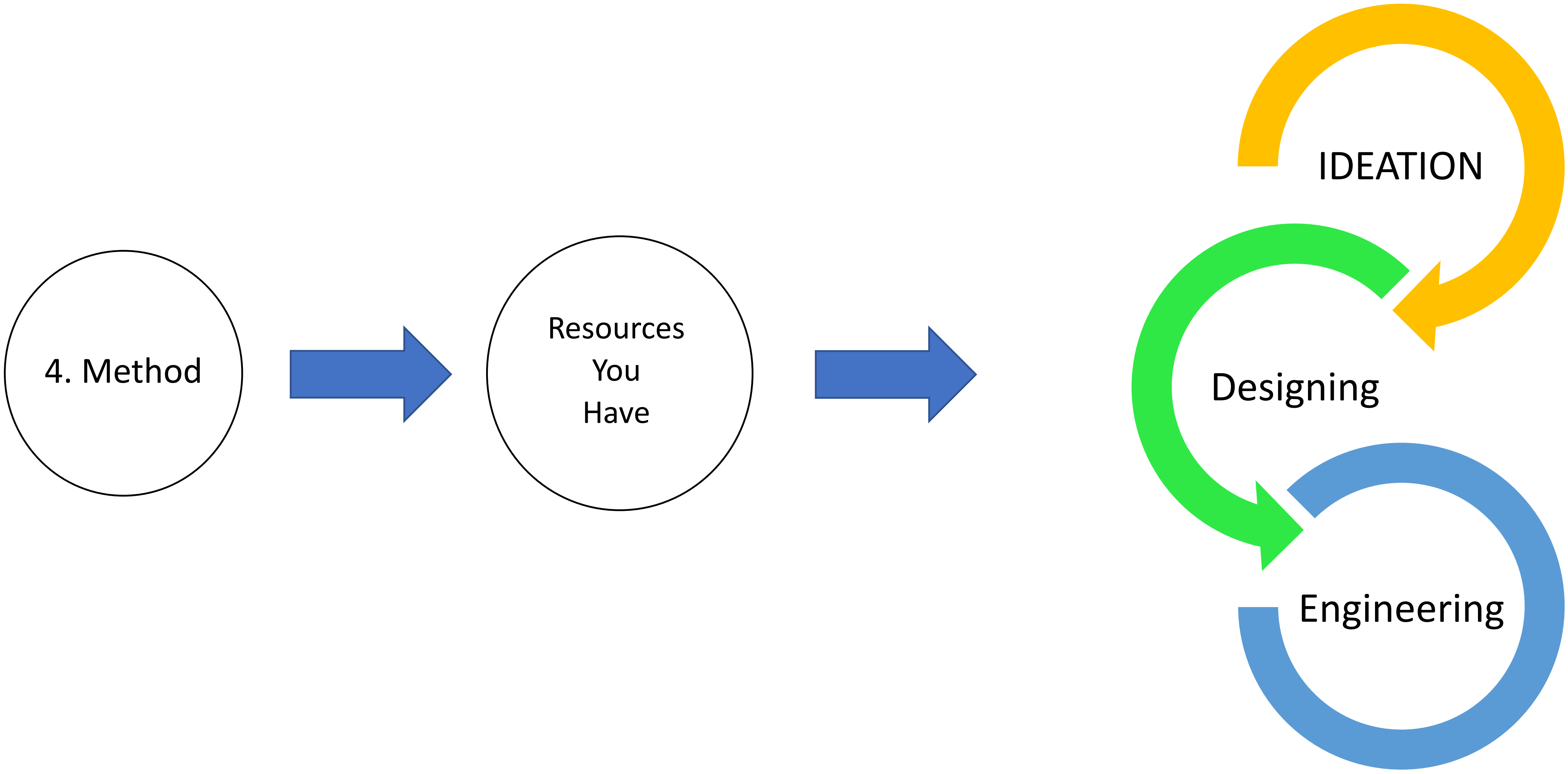
# 3. Solution

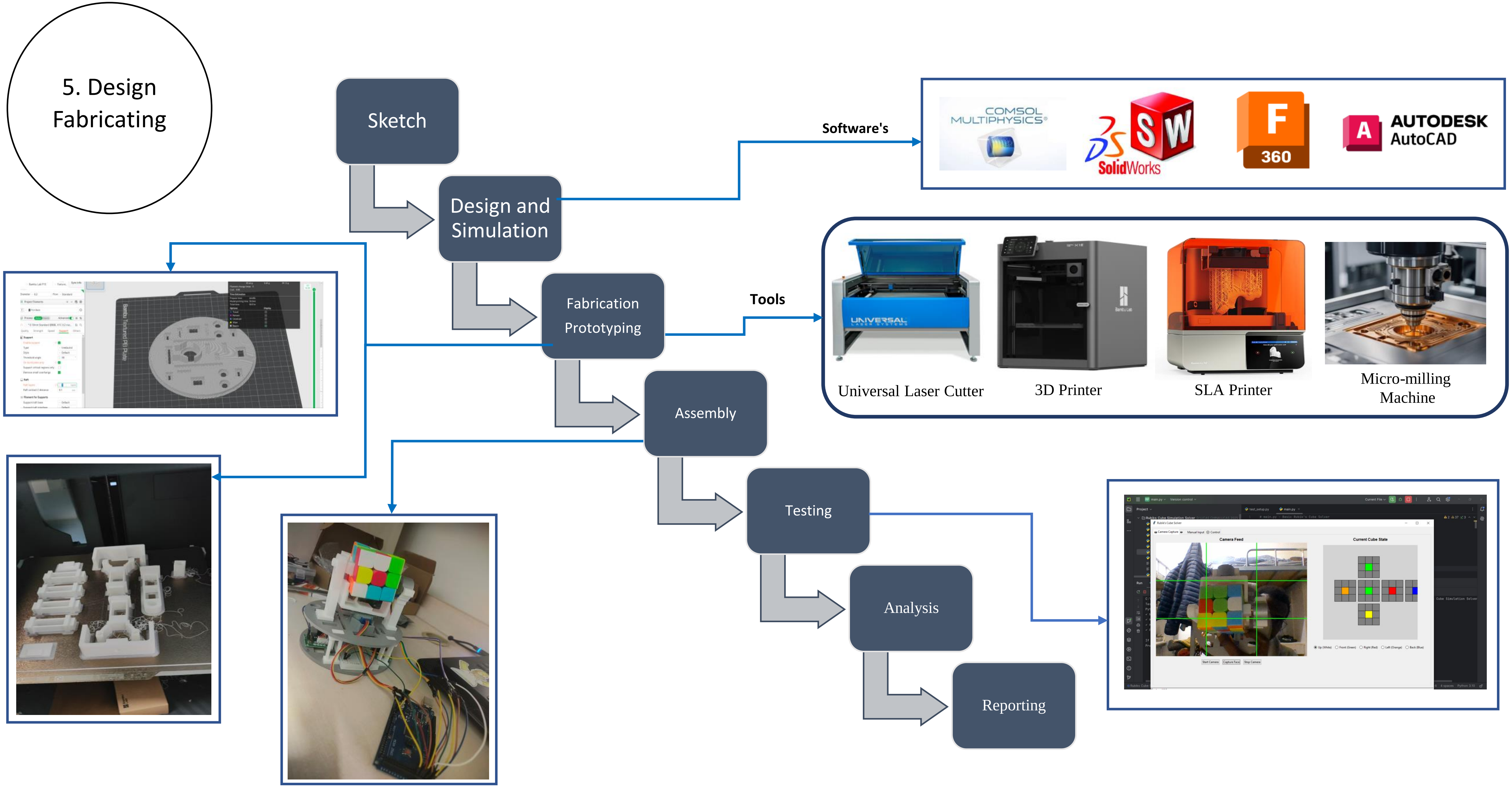
- Our approach leverages 3D printing technology to create durable and customizable parts, combined with readily available electronic components. This enhances the feasibility of replicating the robot in educational settings while providing insights into automation and robotics.



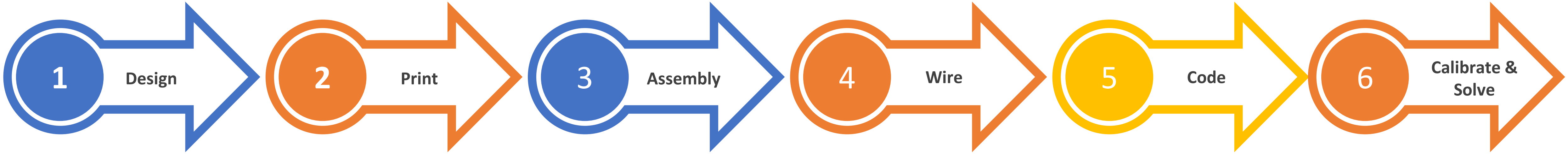


| Total Cost: only 400-500 RMB | |
|---|---|
| Component | Estimated Cost (RMB) |
| 3D Printed Parts (PLA) | ~50 |
| Arduino Mega 2560 | ~120 |
| Stepper Motors (28BYJ-48)x3 | ~60 |
| ULN2003 Drivers | ~30 |
| Power Supply & Cables | ~40 |
| Miscellaneous (Screws, Teflon, etc.) | ~50 |
| Total | ~350-400 RMB |


4. Method
Resources
You
Have
IDEATION
Designing
Engineering


5. Design Fabricating
Sketch
Design and Simulation
Fabrication Prototyping
Assembly
Testing
Analysis
Reporting
Software's
COMSOL MULTIPHYSICS
SolidWorks
360
AUTODESK AutoCAD
Tools
Universal Laser Cutter
3D Printer
SLA Printer
Micro-milling Machine

# Build in 6 Easy Steps




Download The CAD files

**Step 2**

3D print the parts (no supports needed)

**Step 3**

Snap together like LEGO

**Step 4**

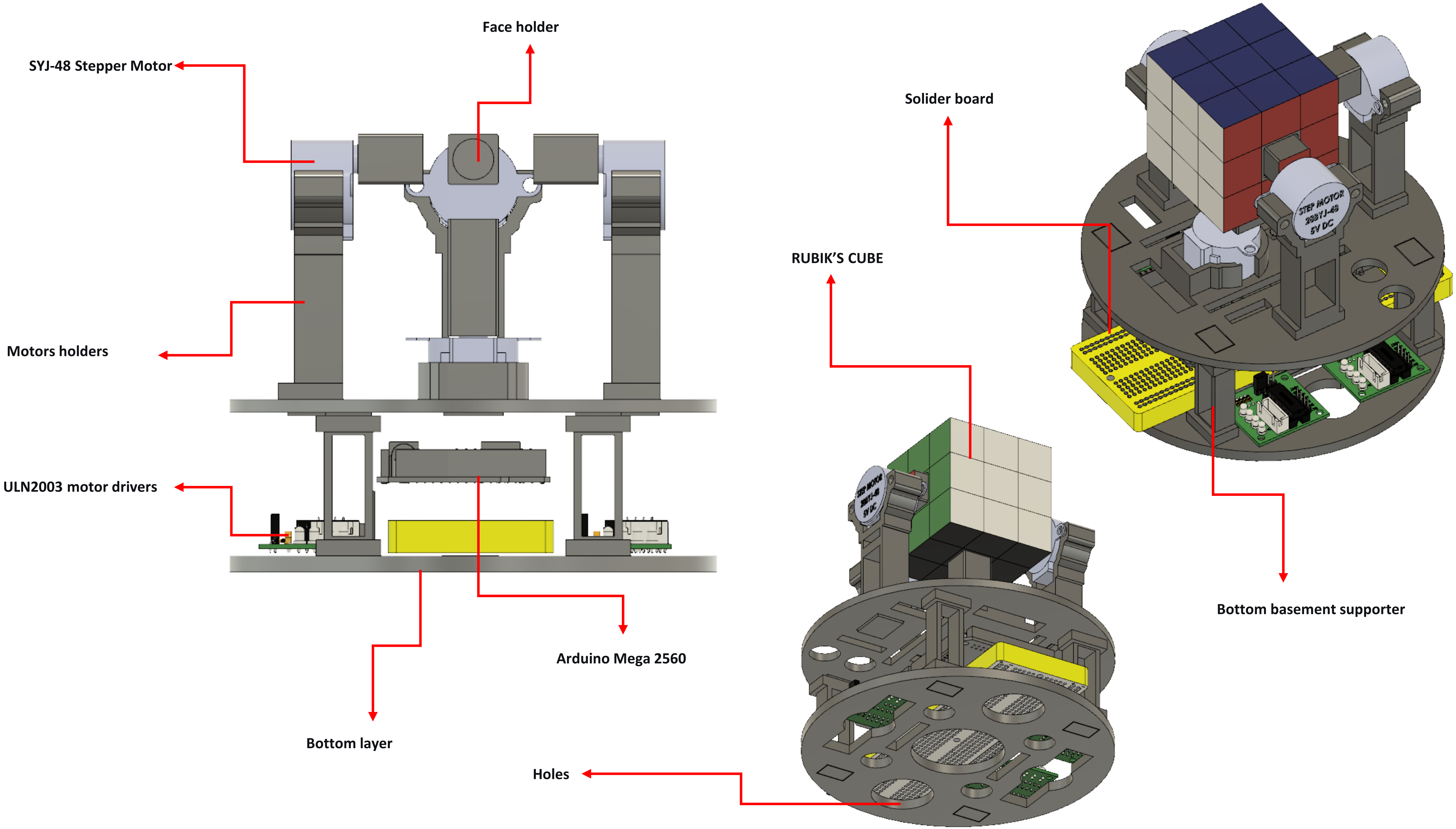
Plug motors into drivers, drivers into Arduino

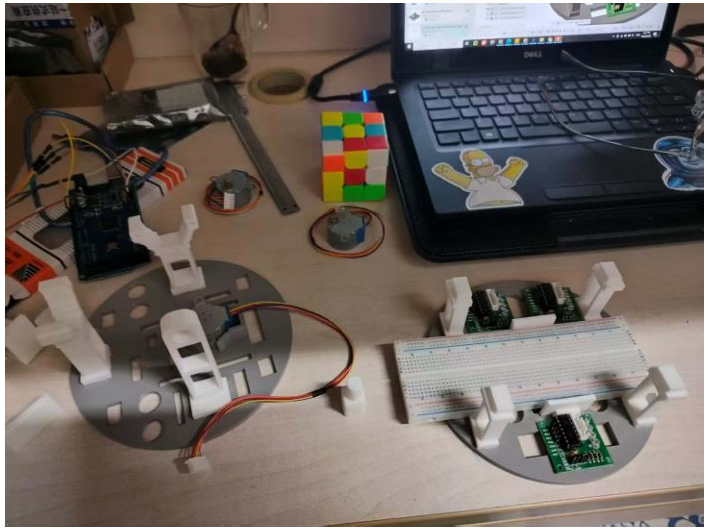
**Step 5**

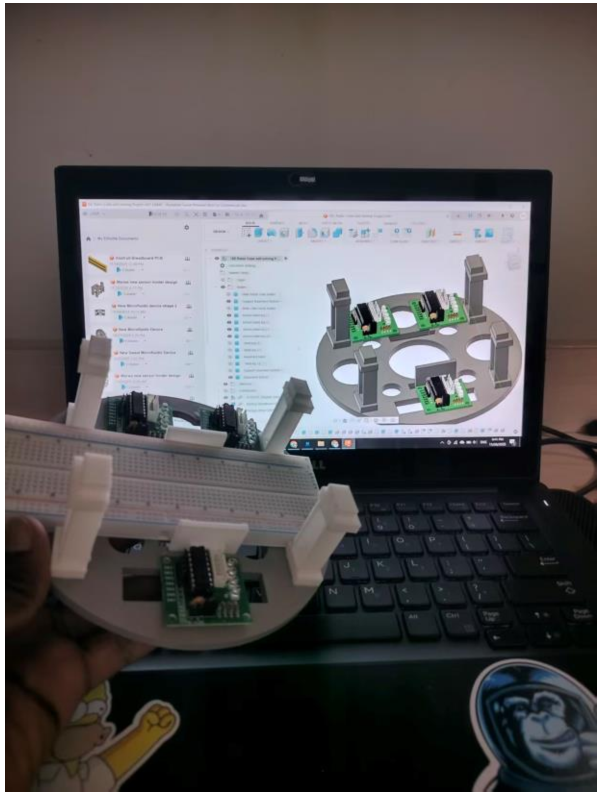
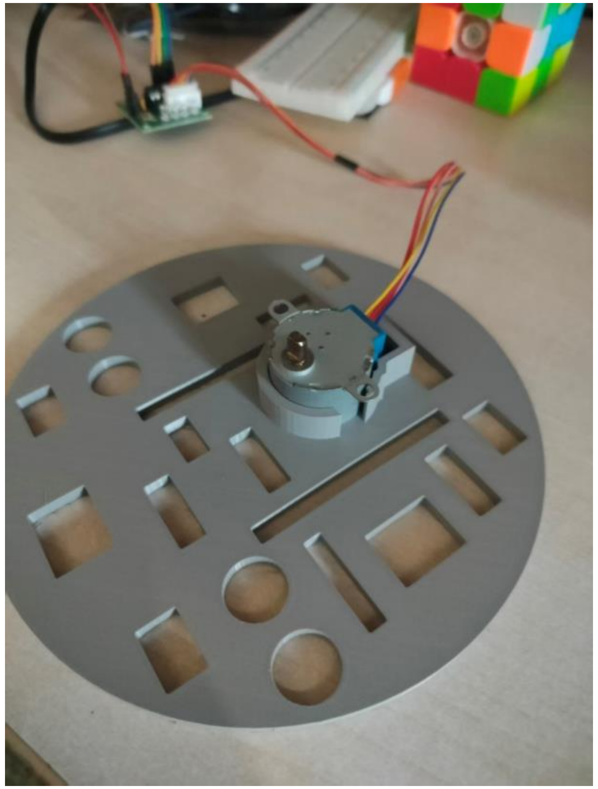
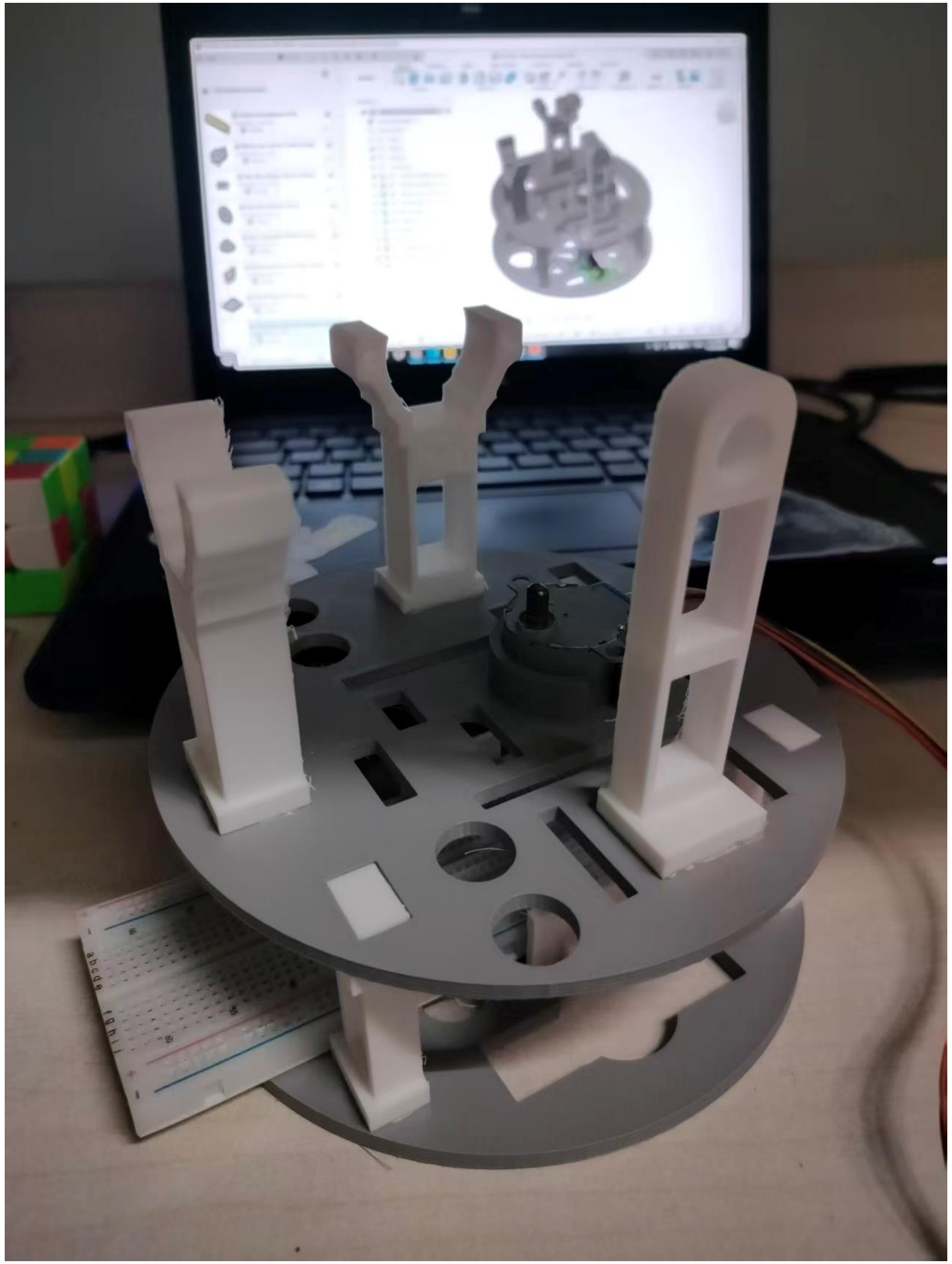
Upload Arduino sketch, run Python GUI

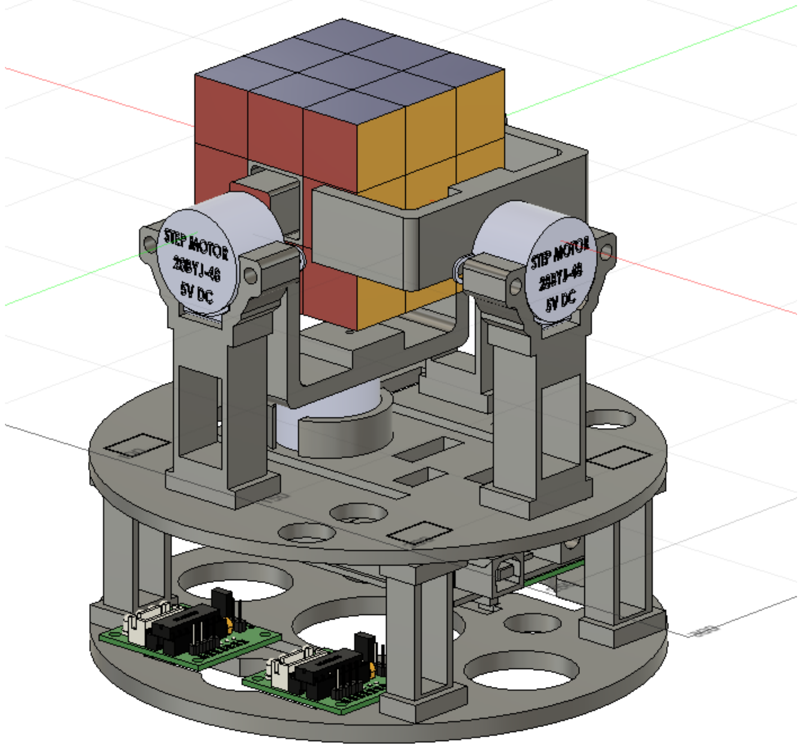
**Step 6**

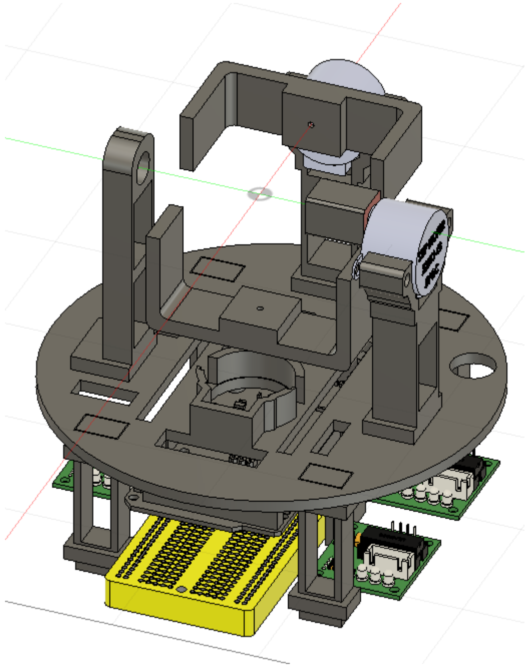
Follow on-screen instructions


Face holder
SYJ-48 Stepper Motor
Solider board
Motors holders
RUBIK'S CUBE
ULN2003 motor drivers
STEP MOTOR
28BYJ-48
5V DC
Bottom basement supporter
Arduino Mega 2560
Bottom layer
Holes





## 6. Challenges & Problems

1. What is the difference between those two models ?
2. Why did we changed it?
3. IS the first design good ?
4. How much time it take to change to this new model ?
5. What could be the challenges, if we strike to the first design ?
6. What we learned from this ?

**Key Challenges:**

1.Mechanical stability with limited components

2.Accurate rotation with low-torque motors

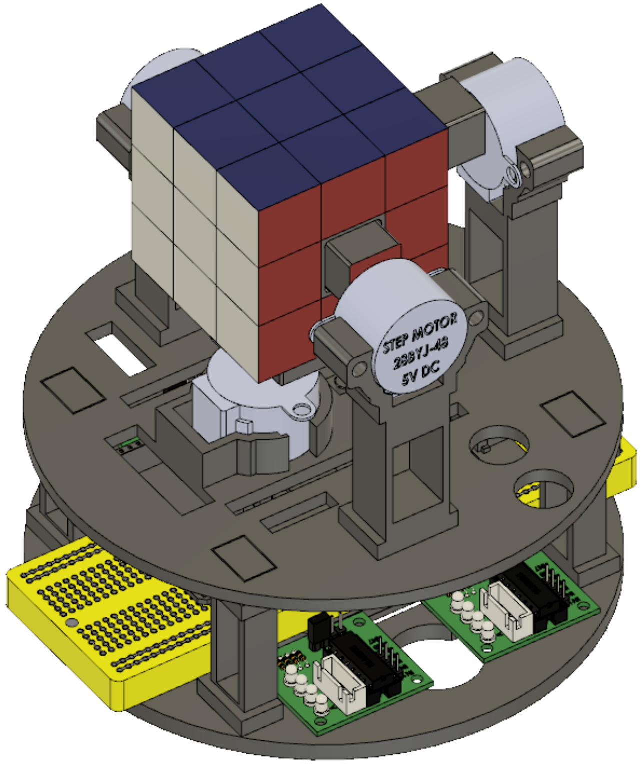
3.Color detection without high-end vision systems

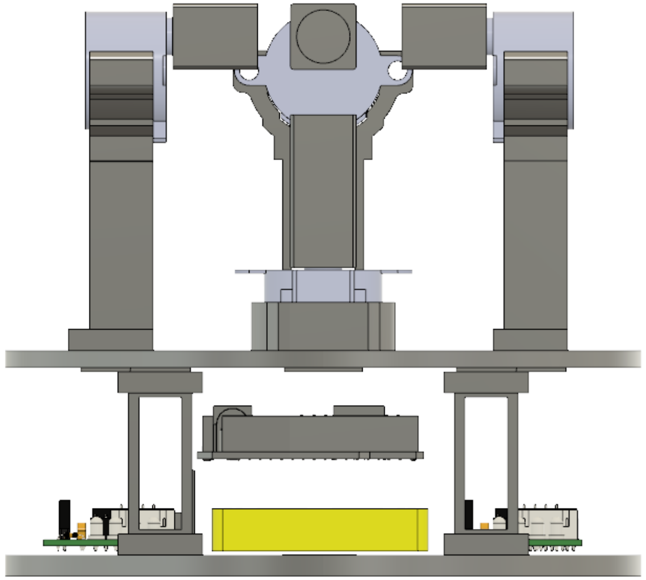
**Mechanical Challenges:**

1. Tolerance issues with 3D printing
2. Motor torque limitations
3. Cumulative alignment errors

**Solutions Implemented:**

1. Iterative CAD redesigns
2. Software position recalibration
3. External power supply addition



## Electronics Implementation (Control System Components):

**Hardware Components:**

1. Arduino Mega 2560 (Main controller)
2. 28BYJ-48 Stepper Motors (3 units)
3. ULN2003 Driver Boards
4. External 5V, 2A Power Supply

**Key Specifications:**

1. Motor torque: 350 gf·cm each
2. Step resolution: 512 steps/90° rotation
3. Serial communication: 9600 baud rate

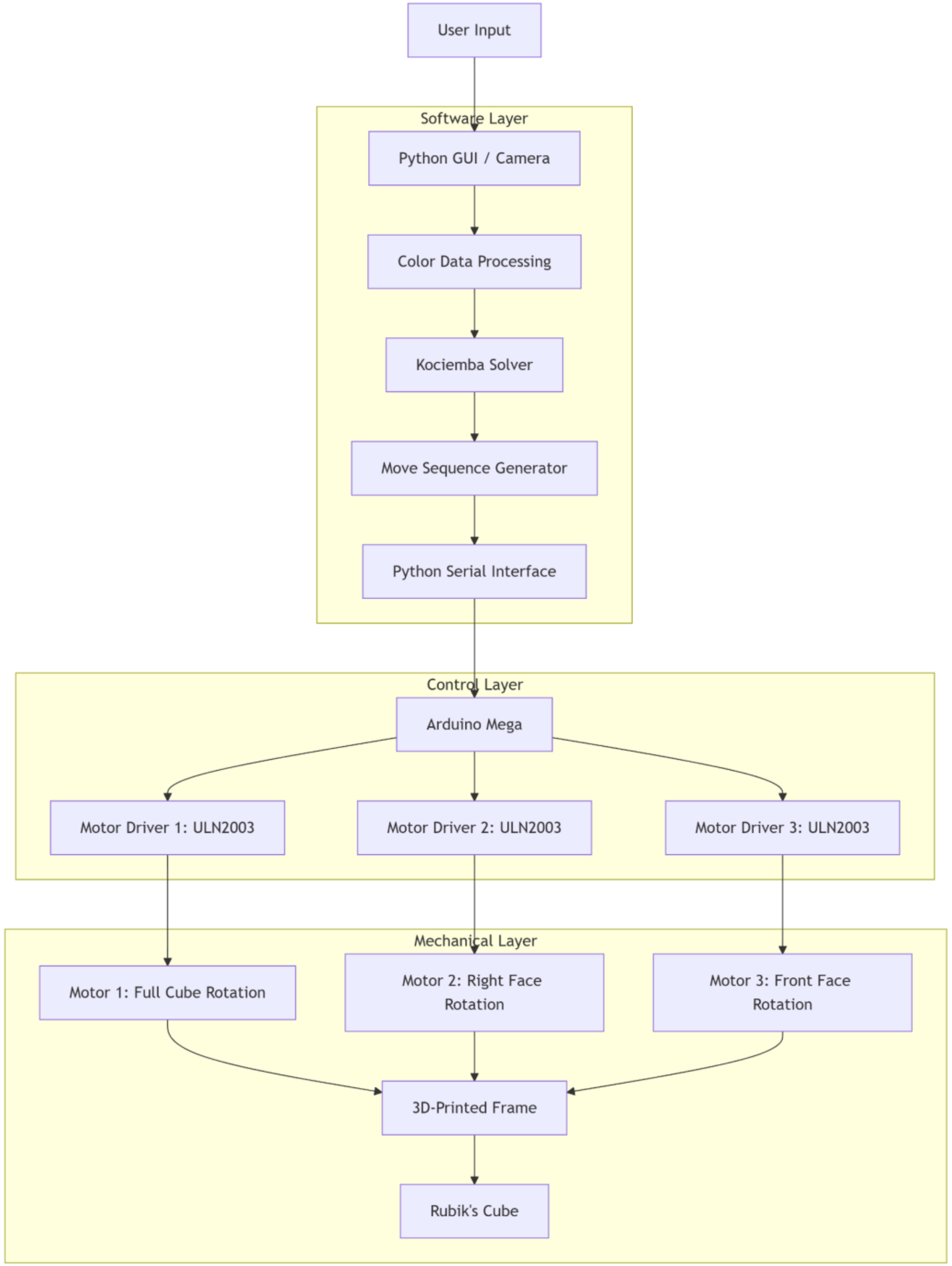
**Modular Input Options:**

1.Color sensor (TCS3200)

2.Manual GUI input

3.Bluetooth camera interface

```
           [ Motor 1 ]
     (Vertical Cube Rotation)
                |
                |
       ┌────────┴────────┐
       |   3D Printed    |
       |   Cube Holder   |
       |                 |
[Motor 3] ── Cube ── [Motor 2]
 (Front Face)        (Bottom Face)
 Rotation             Rotation
```



## 7. Algorithms

**Software Layer:**

1. Python GUI for color input
2. Kociemba solving algorithm
3. Serial communication interface

**Software Challenges:**

1. Timing synchronization
2. Move sequence optimization
3. Color detection accuracy

```
Python GUI              ← Enter colors (2D cube)
     |
     ▼
Cube State Validator
     |
     ▼
Cube Solver (Alg)       ← Generates moves (L R B D etc.)
     |
     ├── Simulation Mode (print + animate)
     |
     └── Arduino Mode (Serial)
              |
              ▼
        Arduino + Motors
```

Fig. Shows GUI and algorithm architecture

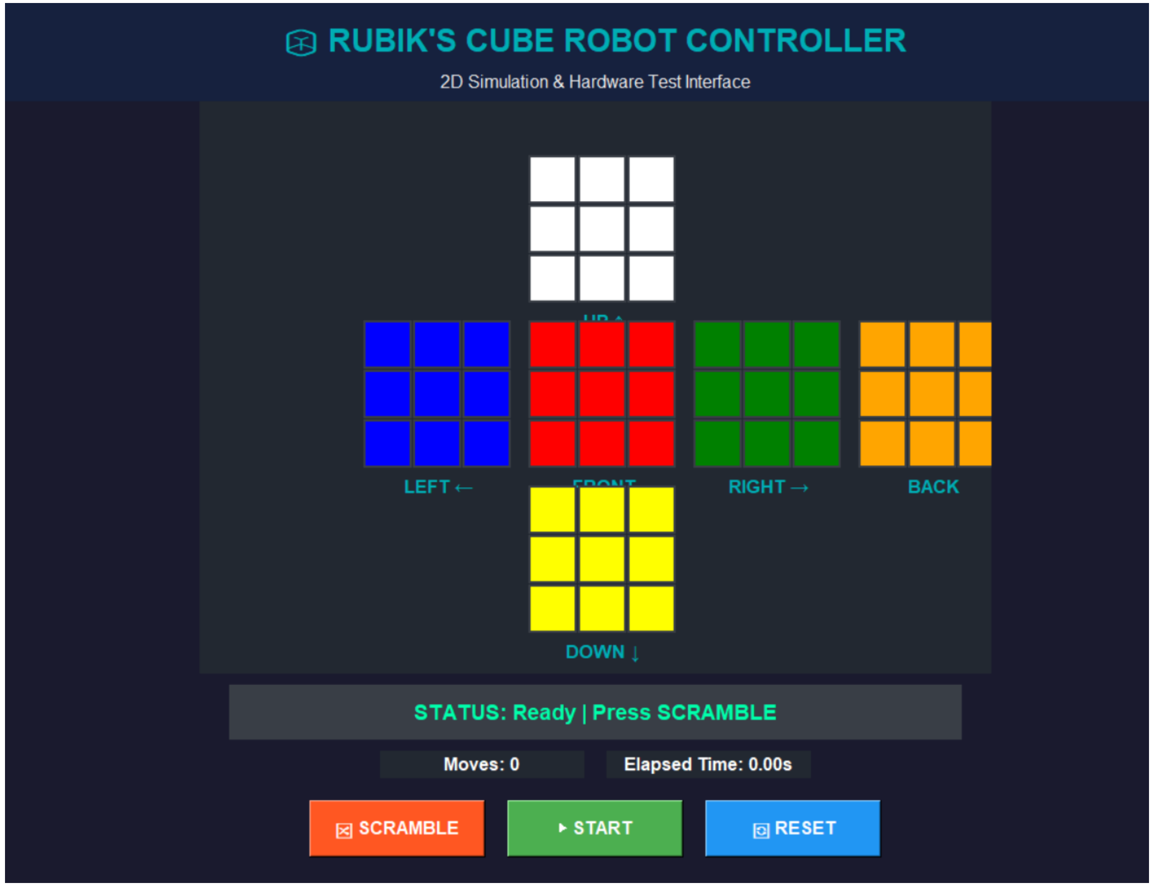



Fig. GUI Control interface

**Can This Algorithm Solve Any Scramble?**

No, this algorithm cannot solve arbitrary Rubik's Cube scrambles. It **Is not a full Rubik's Cube solver**.

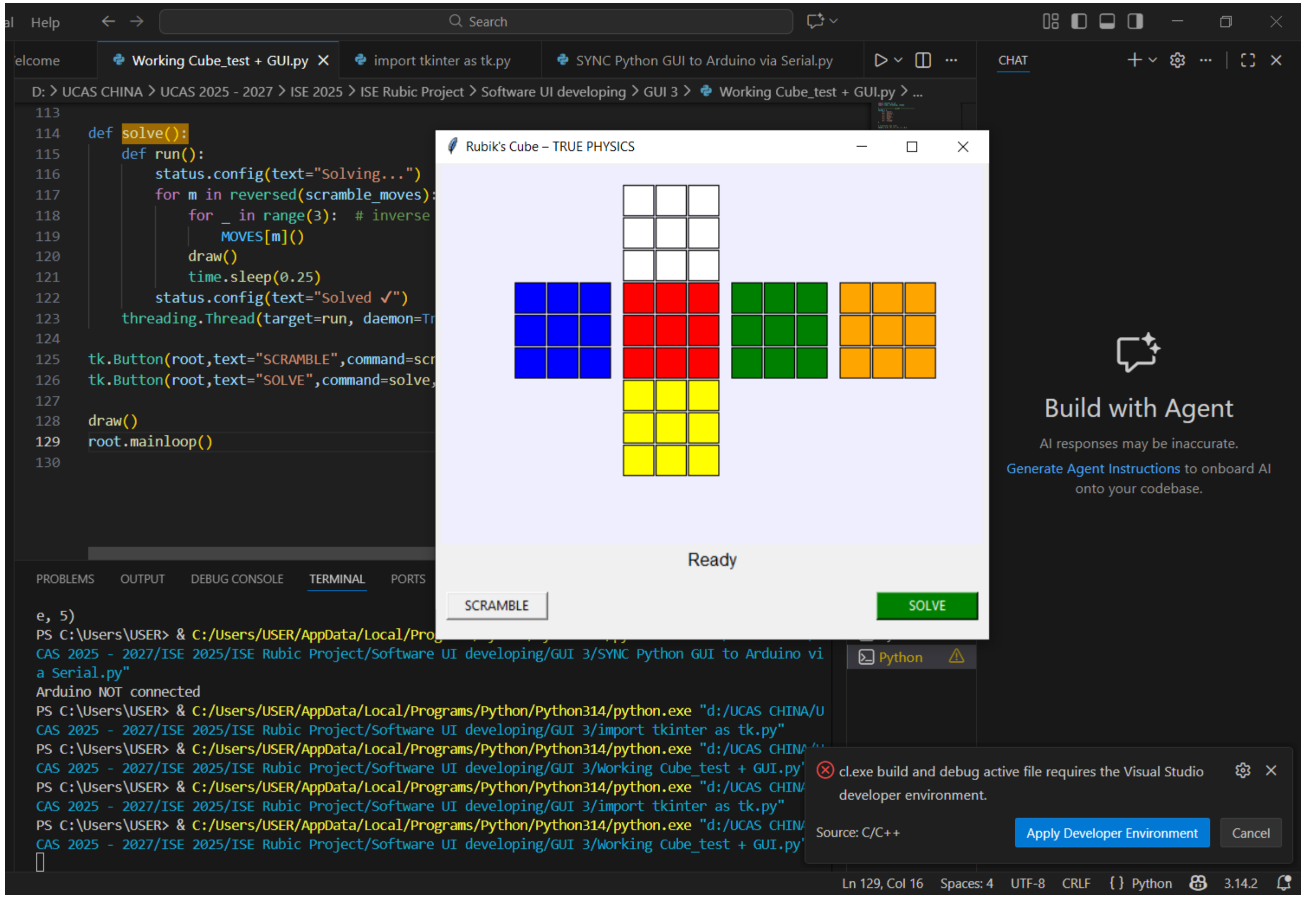
This algorithm is intentionally designed for controlled robotic testing rather than full autonomous cube solving. It guarantees correctness under known scramble conditions and provides a reliable foundation for hardware validation and future intelligent extension

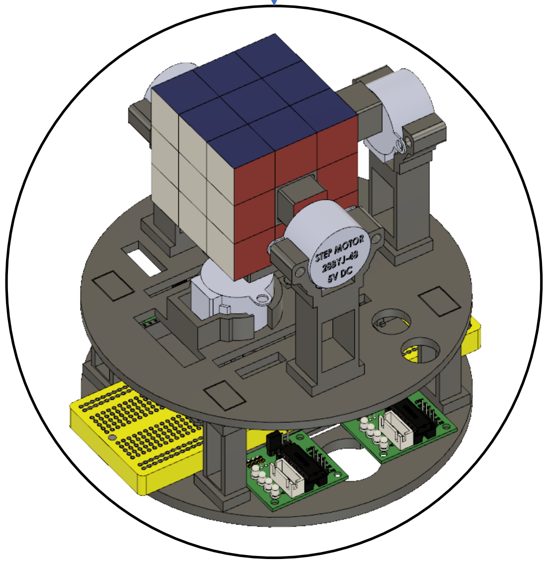
## What This Algorithm CAN DO:

- Correctly reverse robot-generated scrambles
- The cube is scrambled only using known moves (U, R, F)
- The exact scramble sequence is stored
- Solving is done by reversing those moves
- This guarantees a correct solution 100% of the time

## What This Algorithm CAN DO:

- Cannot solve an unknown or hand-scrambled cube
- Cannot solve complex or arbitrary scrambles
- Cannot use camera input
- Not a general Rubik's Cube solver





## How It Works (Simple Version)

**Step 1**

You can simply impress your audience and add a unique zing and appeal to your Presentations. Easy to change colors, photos and Text.

**Step 2**

You can simply impress your audience and add a unique zing and appeal to your Presentations. Easy to change colors, photos and Text.

**Step 3**

You can simply impress your audience and add a unique zing and appeal to your Presentations. Easy to change colors, photos and Text.

**Step 4**

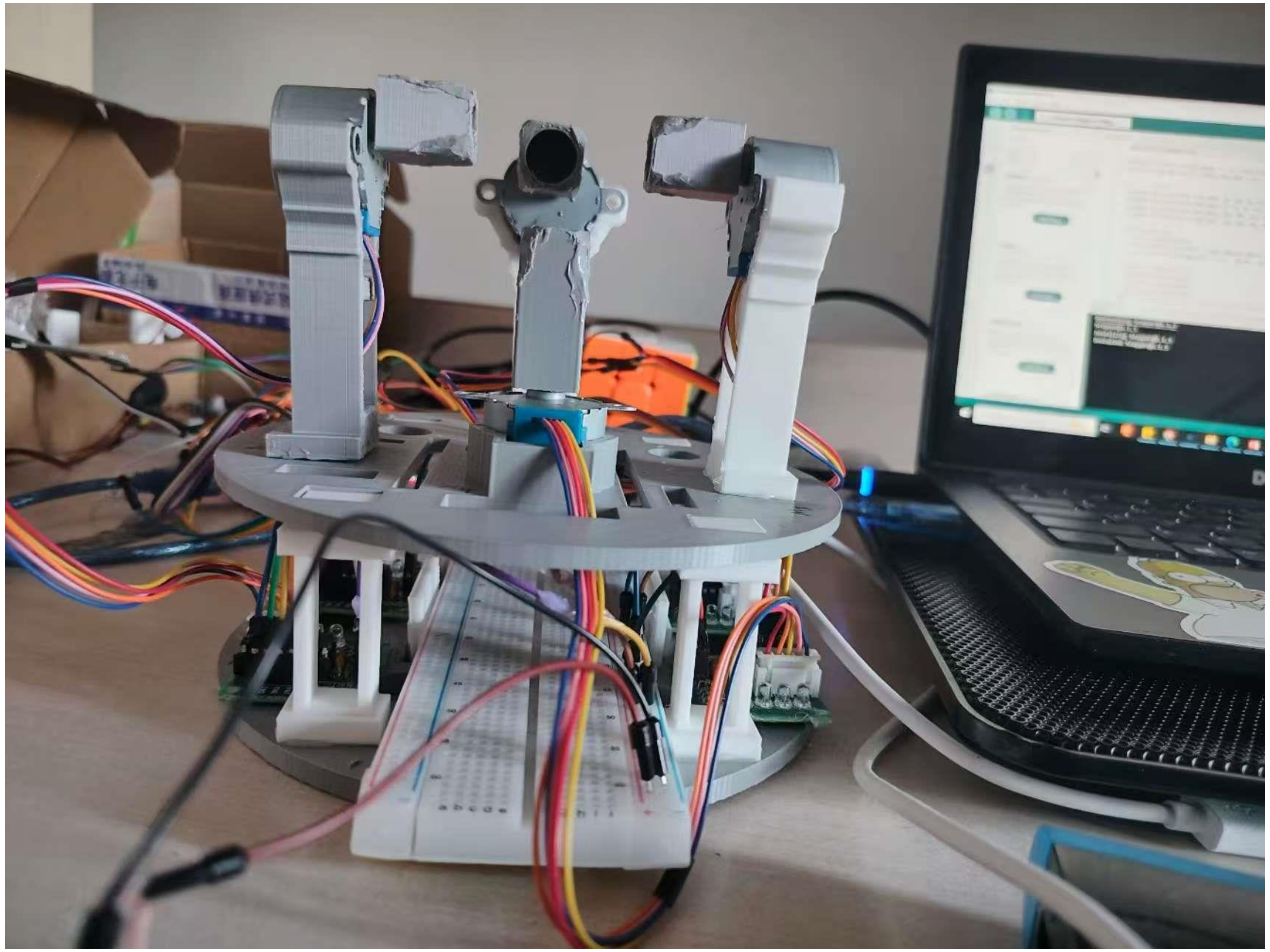
You can simply impress your audience and add a unique zing and appeal to your Presentations. Easy to change colors, photos and Text.

**Step 5**

You can simply impress your audience and add a unique zing and appeal to your Presentations. Easy to change colors, photos and Text.

8. Contributions

# State-of-the-Art Contributions…



Simplicity

Adaptive Sensing Methods

Algorithm Development

Educational Applications

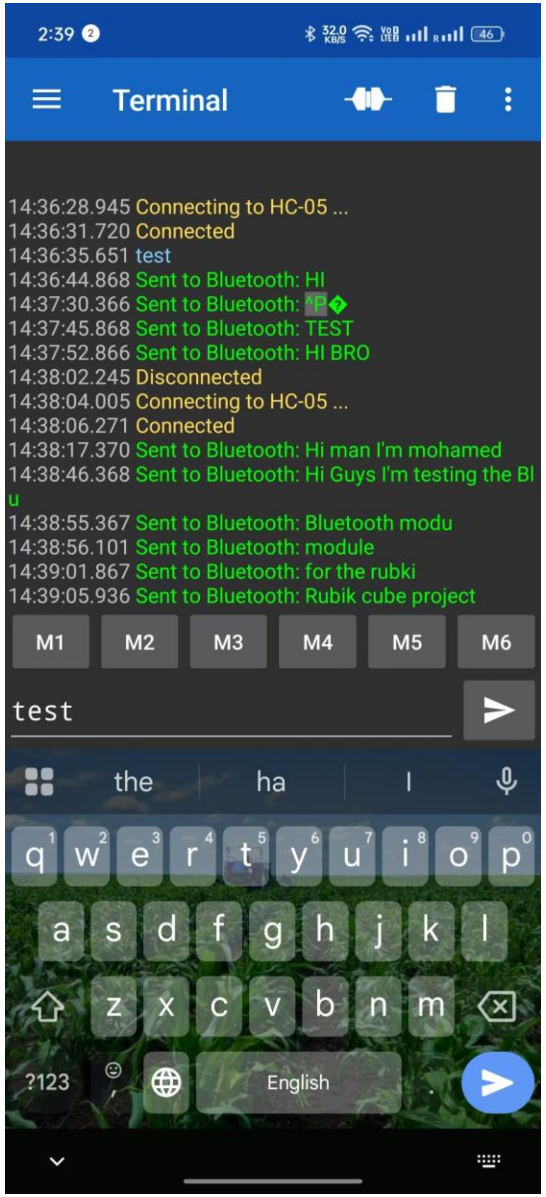
9. Testing and Results

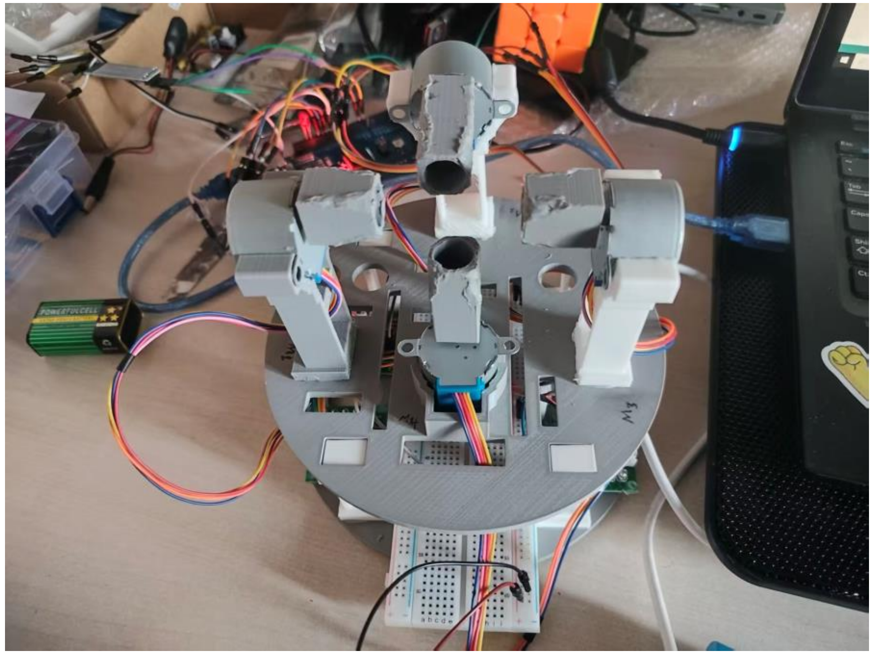
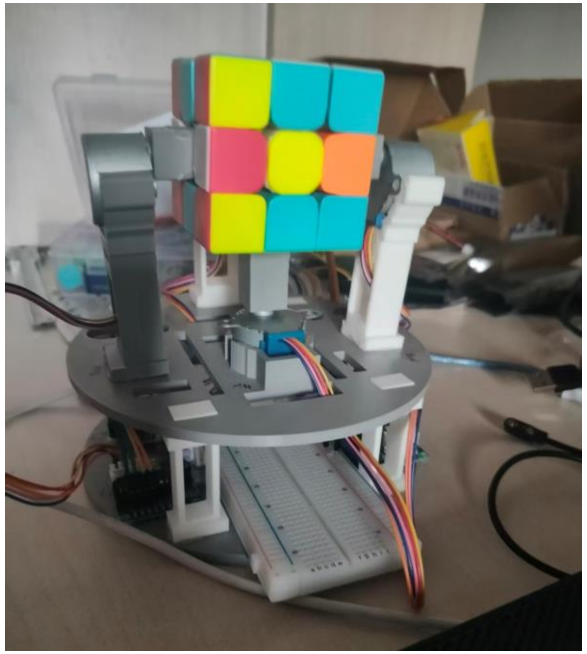
**Testing & Results**

**Testing:**

- Motor alignment verification
- Friction reduction with Teflon tape
- Vibration minimization
- Communication stability at 9600 baud

Video (from your laptop)

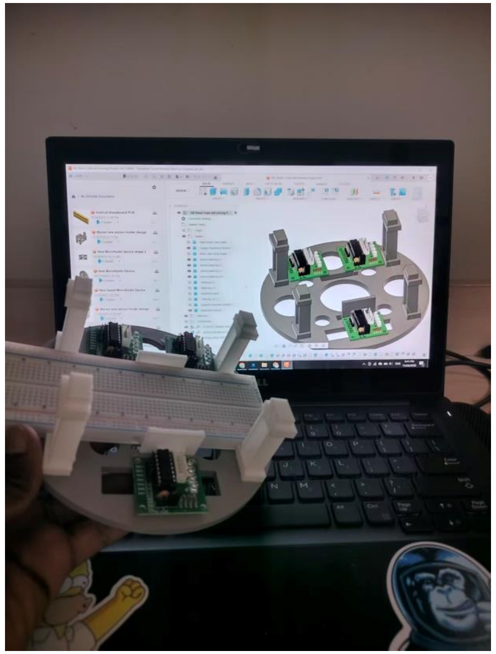


System Testing and data transmission

# 10. Applications

Fig. [YuMi Robot Solving Rubik's Cube - Robotic Gizmos]



Educational Tools: It serves as a practical example for teaching robotics, programming, and STEM concepts.

Robotics Development: The project can be a foundation for more complex robotic systems, encouraging innovation in automation.

Puzzle Solving: Beyond education, the robot can be used in competitions or demonstrations, enhancing interest in robotics and problem-solving.

Research Platform: It offers a platform for testing various algorithms and sensory technologies in robotic applications.

# Learning Outputs

Robotics and automation
Embedded systems programming

Control systems with
Mechanical design
and 3D printing

Problem-solving methodologies

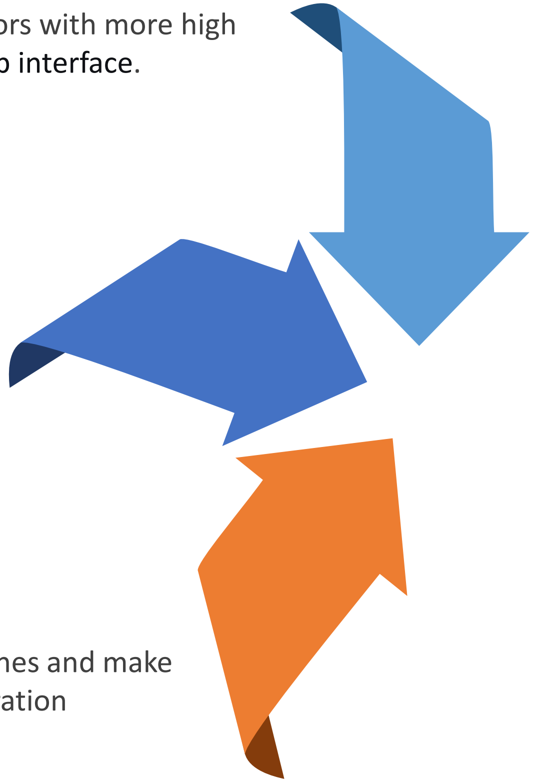
Problem-solving
methodologies

Algorithm
implementation

## 12. Future works

Our approach leverages 3D printing technology to create durable and customizable parts, combined with readily available electronic components. This enhances the feasibility of replicating the robot in educational settings while providing insights into automation and robotics.

Changed and replaced the motors with more high torque and Develop mobile app interface.



Implement reliable camera-based scanning and Improve mechanical rigidity with PETG/ABS

Add automated calibration routines and make the robot Fully autonomous operation

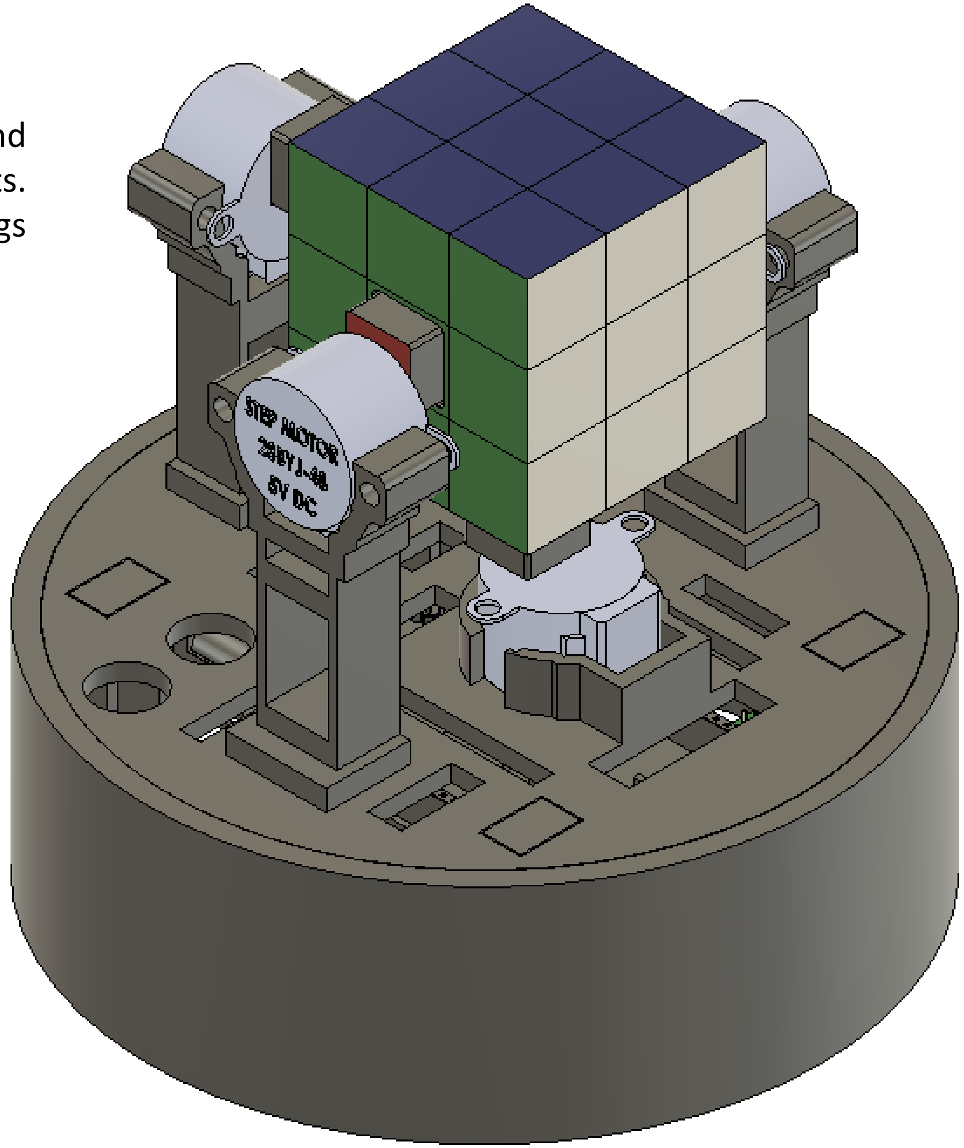
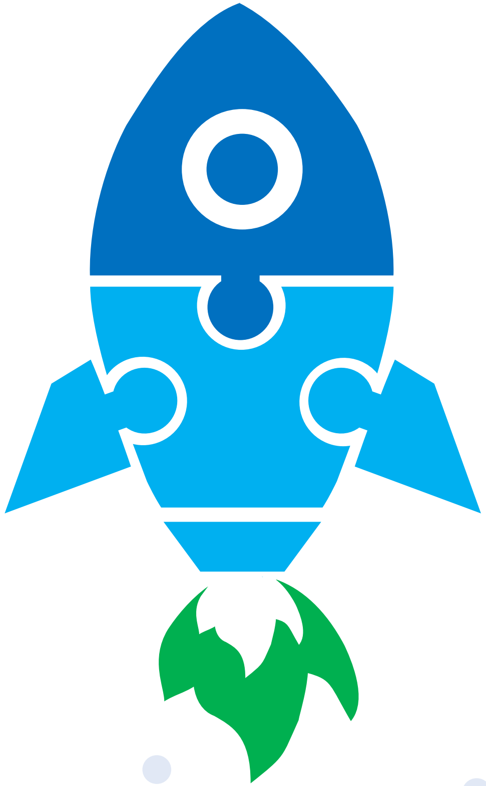



Fig. updated design for the Cube Robot’s

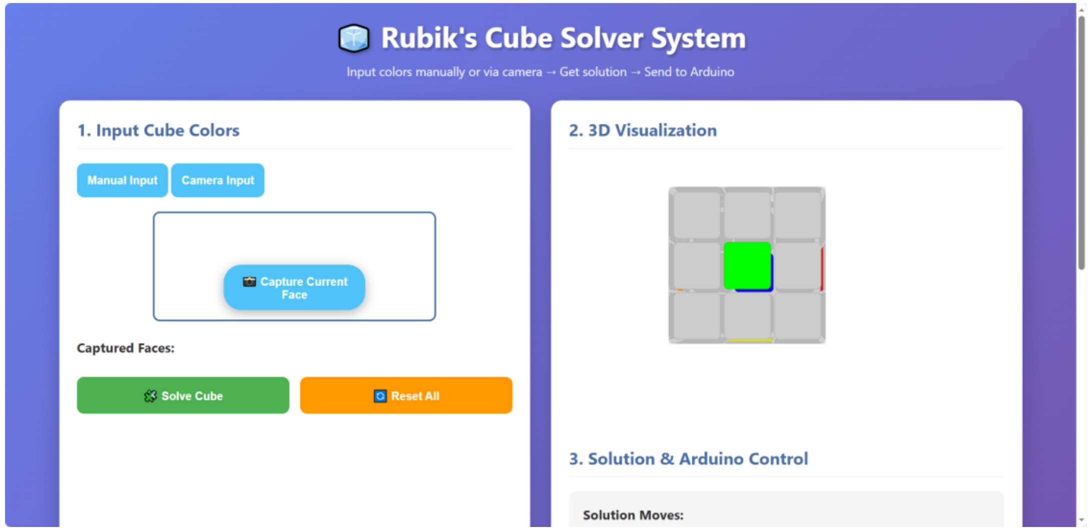
“Try It Yourself – No Experience **Needed!**”

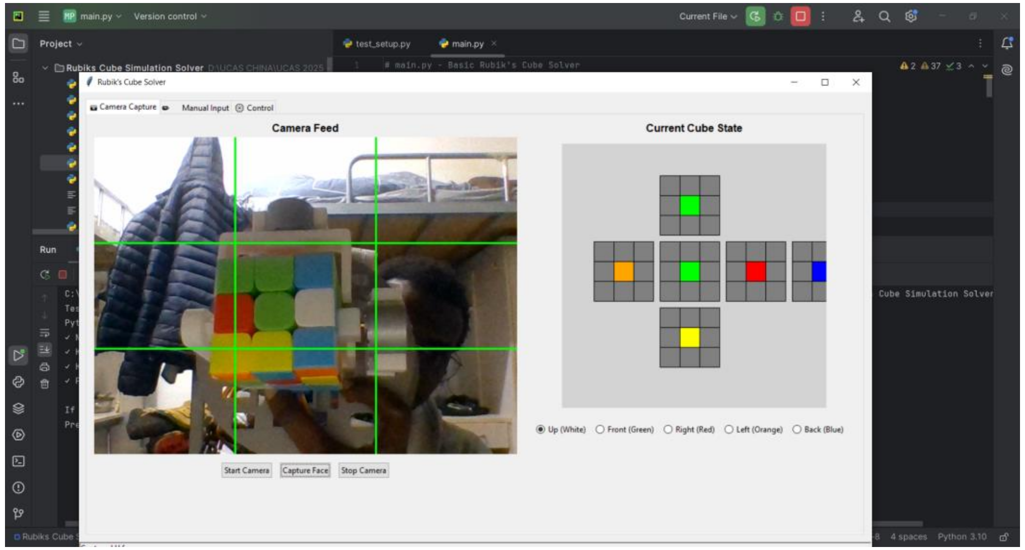
## START NOW……

- Video tutorial available
- All files open-source
- Online community support
- Perfect for school projects

## Future Improvements
Develop and testing different GUI Systems and algorithms




**Rubik's Cube Solver System**
Input colors manually or via camera → Get solution → Send to Arduino




**Rubik's Cube Solver System**
Input colors manually or via camera → Get solution → Send to Arduino